

Web Server Security Assessment Using OWASP Mutillidae II

Assignment — 420-B06-VA Intrusion Testing

Marco Helie

420-B06-VA Intrusion Testing

September 10, 2026

Table of Contents

Objective and Lab Setup	2
Part 1 — Web Server Reconnaissance	3
Part 2 — robots.txt Investigation	4
Part 3 — Information Disclosure Assessment	6
Part 4 — Security Misconfiguration Investigation	7
Part 5 — Attack Research and Proof of Concept	9
Part 6 — Patch Management Assessment	11
Part 7 — Web Server Hardening Plan	12
Conclusion	13

Objective and Lab Setup

The objective of this assignment was to act as a Junior Penetration Tester and perform a security assessment of a web server hosting the OWASP Mutillidae II application. The goal was not to exploit vulnerabilities, but to identify information exposed by the server, investigate potential security weaknesses, research attack techniques, and develop remediation recommendations.

The lab environment was set up using Docker on a Kali Linux virtual machine, using the following commands:

```
docker pull citizenstig/nowasp
docker run -d -p 80:80 citizenstig/nowasp
```

The application was then accessed at <http://localhost/>. On the first visit, the database had to be initialized using the "setup/reset the DB" link, and the security level was set to 0, as intended for this training application.

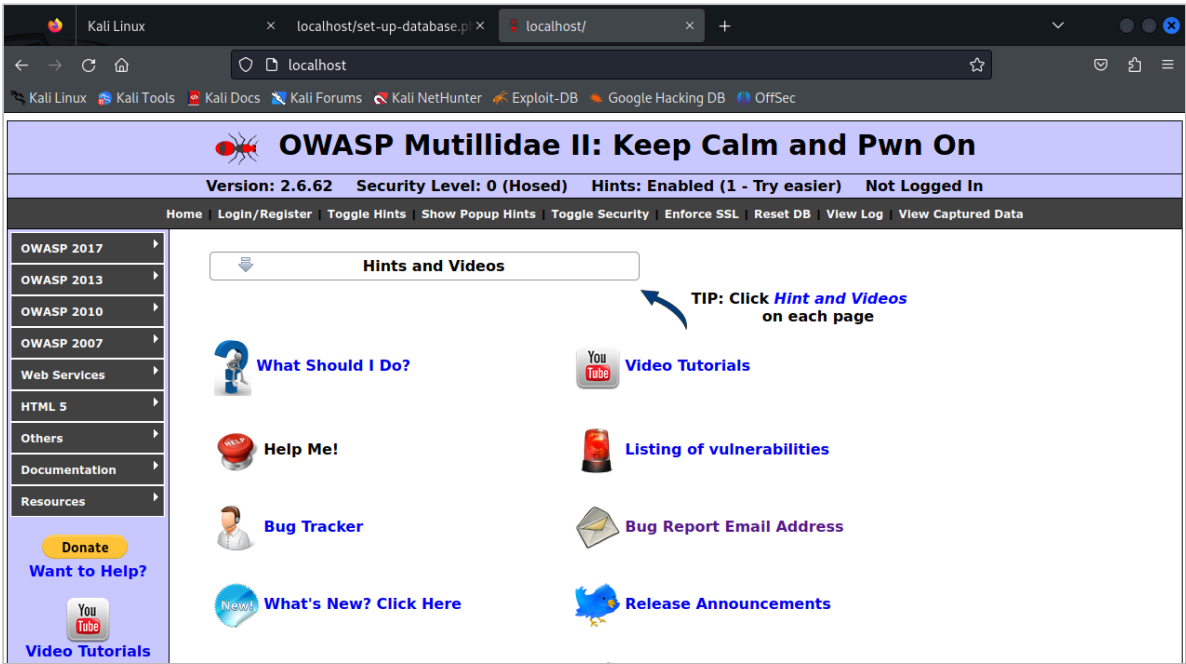


Figure 1 — OWASP Mutillidae II homepage running locally (Version 2.6.62, Security Level 0)

Part 1 — Web Server Reconnaissance

Reconnaissance was performed against the server using three tools: `curl` to view the HTTP response headers, `nmap` to scan open TCP ports and identify services, and `whatweb` to fingerprint the web technologies in use.

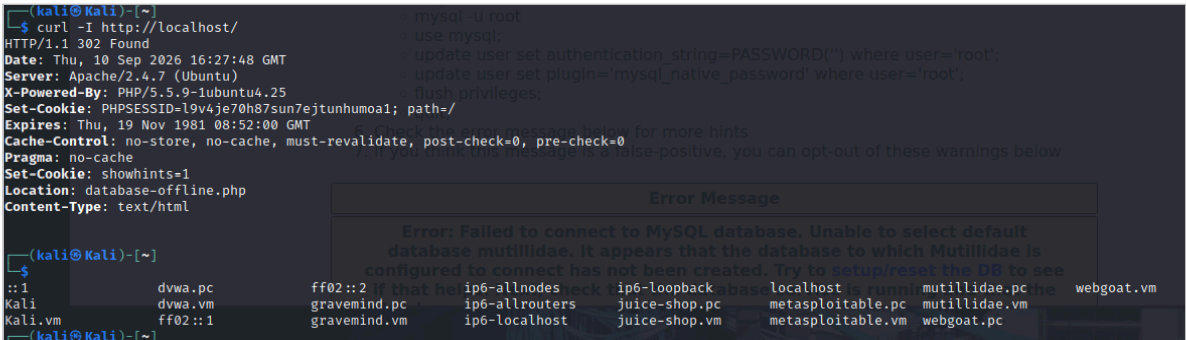


Figure 2 — HTTP response headers obtained with `curl -I http://localhost/`

```
(kali㉿kali)-[~]
└─$ nmap -sV localhost
Starting Nmap 7.94 ( https://nmap.org ) at 2026-09-10 16:35 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00024s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 998 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.3p2 Debian 1 (protocol 2.0)
80/tcp    open  http      Apache httpd 2.4.7 ((Ubuntu))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.10 seconds

(kali㉿kali)-[~]
└─$ whatweb http://localhost/
http://localhost/ [200 OK] Apache[2.4.7], Cookies[PHPSESSID,showhints], HTTPServer[Ubuntu Linux][Apache/2.4.7 (Ubuntu)], IP[::1], JQuery, PHP[5.5.9-1ubuntu4.25], PoweredBy[HTTP], Script[text/javascript], UncommonHeaders[logged-in-user], X-Powered-By[PHP/5.5.9-1ubuntu4.25], X-XSS-Protection[0]
```

Figure 3 — Port scan (`nmap -sV localhost`) and technology fingerprinting (`whatweb`)

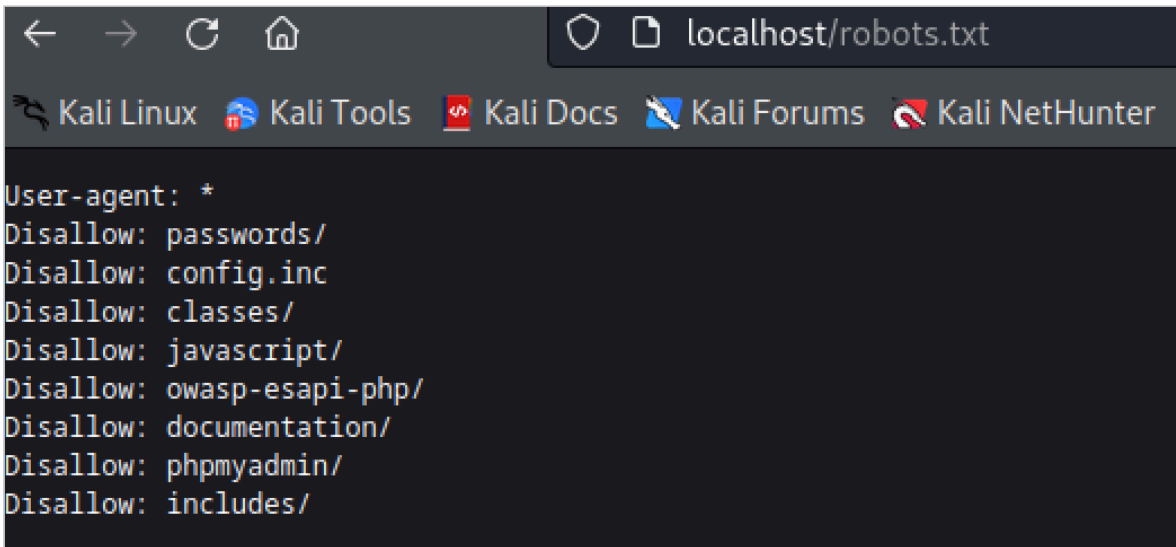
Results

Item	Result
Web server software	Apache
Server version	2.4.7
Operating system	Ubuntu (Linux)
Open TCP ports	22 (SSH — OpenSSH 9.3p2), 80 (HTTP — Apache httpd 2.4.7)
Web technologies	PHP 5.5.9, jQuery, JavaScript, PHPSESSID cookies
HTTP response headers	Server: Apache/2.4.7 (Ubuntu) , X-Powered-By: PHP/5.5.9-1ubuntu4.25 , Set-Cookie (PHPSESSID), Cache-Control

Analysis: The server openly reveals its exact software versions and operating system to any visitor who sends a simple request. This is valuable information for an attacker: Apache 2.4.7 dates back to 2013 and PHP 5.5 has been end-of-life since 2016, meaning both have many publicly known vulnerabilities (CVEs) with ready-made exploits. An attacker can simply look up these exact versions and pick a working attack.

Part 2 — robots.txt Investigation

The robots.txt file was requested directly at `http://localhost/robots.txt` .



```
User-agent: *
Disallow: passwords/
Disallow: config.inc
Disallow: classes/
Disallow: javascript/
Disallow: owasp-esapi-php/
Disallow: documentation/
Disallow: phpmyadmin/
Disallow: includes/
```

Figure 4 — Contents of robots.txt

1. Does robots.txt exist?

Yes. The file exists and is readable by anyone at `http://localhost/robots.txt`.

2. What directories or files are listed?

The file lists: `passwords/`, `config.inc`, `classes/`, `javascript/`, `owasp-esapi-php/`, `documentation/`, `phpmyadmin/`, and `includes/`.

3. Could the information help an attacker?

Yes. robots.txt is a public file readable by anyone, not just search engines. Here it acts like a map pointing directly to the most sensitive areas of the server — a folder literally named `passwords/` and the `phpmyadmin/` database administration panel. The "Disallow" rule only asks search engines not to index these paths; it provides zero access protection.

4. What information should not be publicly disclosed?

The existence and location of password storage, database administration tools (phpMyAdmin), configuration files (config.inc), and internal application source code folders (classes/, includes/) should never be advertised publicly.

5. What recommendations would you provide?

Do not rely on robots.txt to hide sensitive content — it offers no protection, only a hint to attackers. Sensitive directories should be protected with real access controls (authentication, .htaccess rules, or IP allow-listing),

should be moved outside the web root where possible, and sensitive entries should be removed from robots.txt entirely.

Part 3 — Information Disclosure Assessment

Server Response Headers

As shown in Figure 2, the response headers reveal `Server: Apache/2.4.7 (Ubuntu)` and `X-Powered-By: PHP/5.5.9-1ubuntu4.25`. This tells any visitor the exact web server, version, operating system, and PHP version — making it trivial to search for known exploits matching those exact versions.

Error Messages

When the application could not reach its database, it displayed a detailed error page to any visitor.

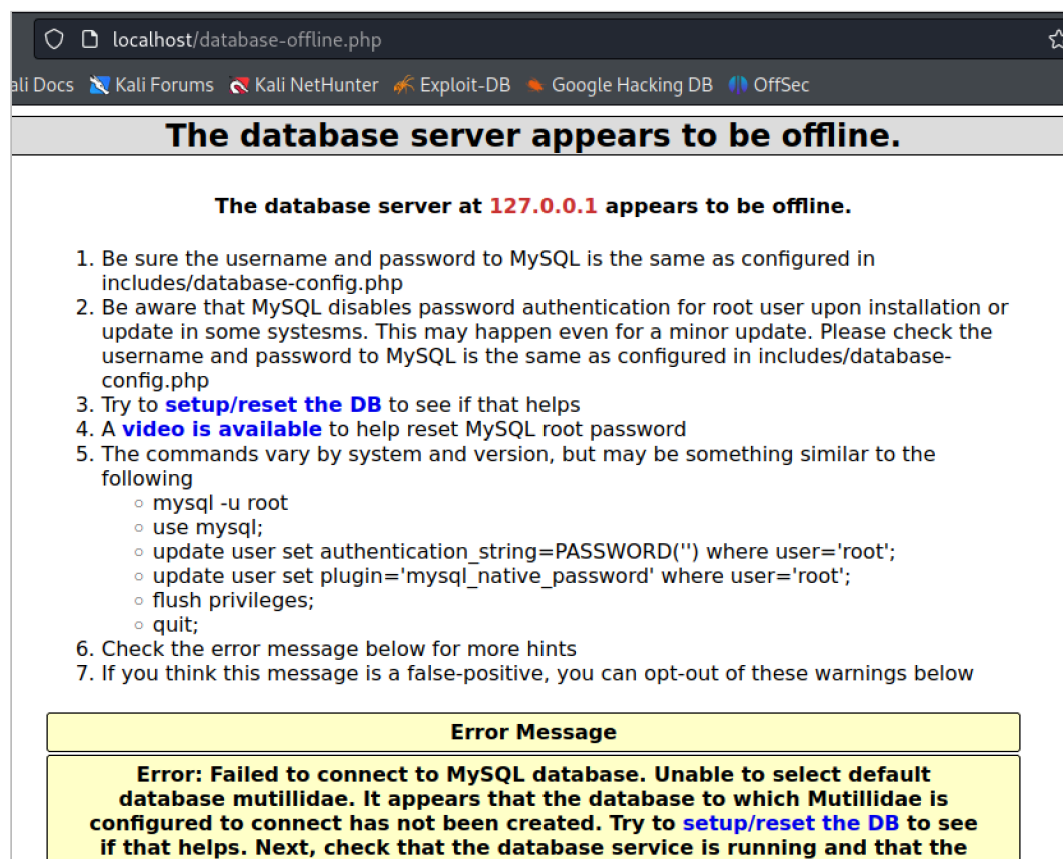


Figure 5 — Verbose error page revealing internal details

This single error page reveals: the internal path of the configuration file (`includes/database-config.php`), the database name (`mutillidae`), the database server address (`127.0.0.1`), the fact that MySQL is used, and even suggests SQL commands for resetting the root password. On a real production server, this

on this page; however, the source confirms the technologies in use and reveals the internal directory structure (e.g., `./images/`).

```
868 </div> <br style="clear: left;" /> -->
869 </div>
870 <div class="label" style="text-align: center;">
871 <form action="https://www.paypal.com/cgi-bin/webscr" method="post" target="_blank">
872 <input type="hidden" name="cmd" value="$$_click">
873 <input type="hidden" name="hosted_button_id" value="45R3YXENU975">
874 <input type="image" src="https://www.paypalobjects.com/en_US/i/btn/btn_donate_LG.gif" border="0" name="submit" alt="PayPal - The safer, easier way to pay online!">
875 </form>
876 <span style="color: blue;">Want to Help?</span>
877 </div>
878 <div class="label" style="text-align: center;">
879 <a href="http://www.youtube.com/user/webpwnized" style="white-space: nowrap;" target="_blank">
880 
881 <br/>
882 Video Tutorials
883 </a>
884 </div>
885 <div class="label" style="text-align: center;">
886 <a href="https://twitter.com/webpwnized" target="_blank">
```

Figure 6 — Reviewing the page source for comments and hidden information

Most importantly, the assessment found that the directory `/passwords/` — disclosed in `robots.txt` — exposes a file named `accounts.txt` containing 23 usernames and passwords stored in plain text (see Part 4). This is the most serious information disclosure identified during the assessment.

Part 4 — Security Misconfiguration Investigation

Finding 1: Directory listing enabled and exposed password file

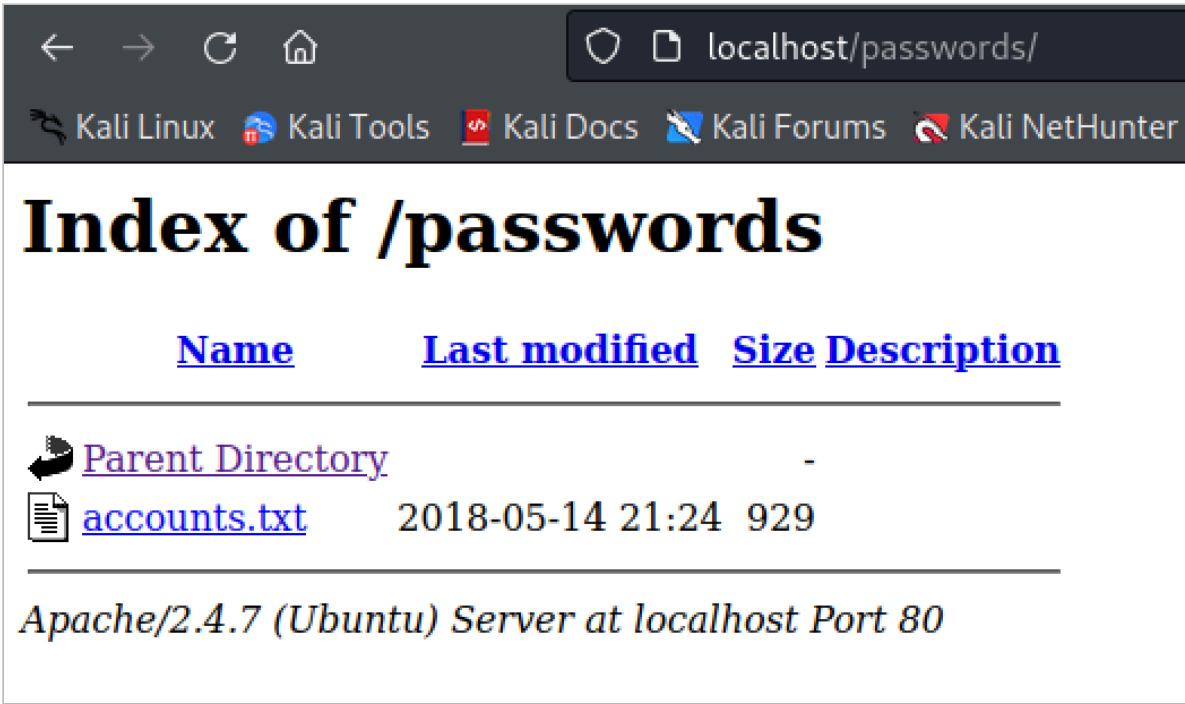
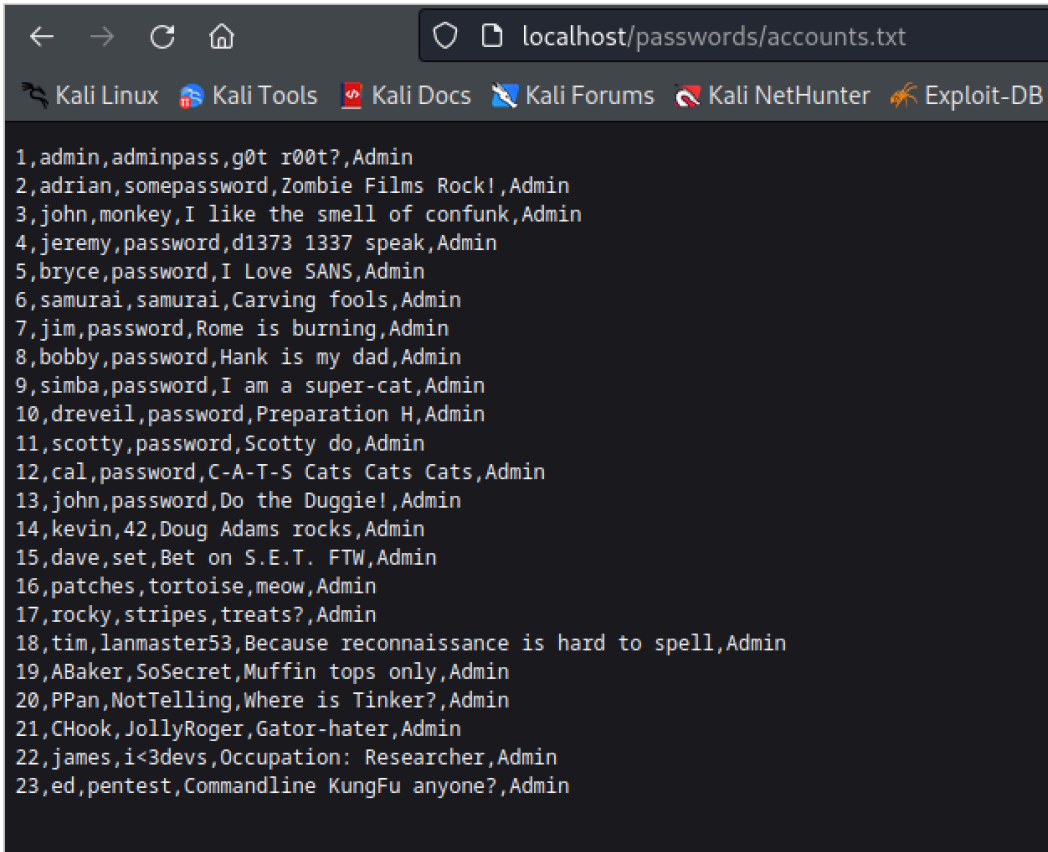


Figure 7 — Directory listing of `/passwords/` (note the server version banner at the bottom)



```
localhost/passwords/accounts.txt

1,admin,adminpass,g0t r00t?,Admin
2,adrian,somepassword,Zombie Films Rock!,Admin
3,john,monkey,I like the smell of confunk,Admin
4,jeremy,password,d1373 1337 speak,Admin
5,bryce,password,I Love SANS,Admin
6,samurai,samurai,Carving fools,Admin
7,jim,password,Rome is burning,Admin
8,bobby,password,Hank is my dad,Admin
9,simba,password,I am a super-cat,Admin
10,dreveil,password,Preparation H,Admin
11,scotty,password,Scotty do,Admin
12,cal,password,C-A-T-S Cats Cats Cats,Admin
13,john,password,Do the Duggie!,Admin
14,kevin,42,Doug Adams rocks,Admin
15,dave,set,Bet on S.E.T. FTW,Admin
16,patches,tortoise,meow,Admin
17,rocky,stripes,treats?,Admin
18,tim,lanmaster53,Because reconnaissance is hard to spell,Admin
19,ABaker,SoSecret,Muffin tops only,Admin
20,PPan,NotTelling,Where is Tinker?,Admin
21,CHook,JollyRoger,Gator-hater,Admin
22,james,i<3devs,Occupation: Researcher,Admin
23,ed,pentest,Commandline KungFu anyone?,Admin
```

Figure 8 — accounts.txt — 23 username/password pairs in plain text

- **Issue:** Browsing to `/passwords/` lists the directory contents, and `accounts.txt` can be opened to reveal 23 usernames and passwords in plain text (including `admin / adminpass`).
- **How an attacker could use it:** Log in immediately as any user or administrator, and reuse the passwords in credential-stuffing attacks against other systems.
- **Possible impact:** Complete compromise of all user accounts and potentially the entire application and server.
- **Mitigation:** Disable directory listing (Options `-Indexes` in Apache), never store credential files inside the web root, and store passwords hashed rather than in plain text.

Finding 2: Publicly accessible phpMyAdmin panel

- **Issue:** `/phpmyadmin/` is reachable by anyone. The page reveals the MySQL version (5.5.60), the database user (`root@localhost`), and the phpMyAdmin version (3.5.2.2 — heavily outdated).
- **How an attacker could use it:** Brute-force the database login, or exploit one of the many known vulnerabilities of phpMyAdmin 3.x to take over the database directly.
- **Possible impact:** Theft, modification, or deletion of all application data.
- **Mitigation:** Restrict access to phpMyAdmin by IP allow-listing and strong authentication, or remove it from the web server entirely; upgrade to a supported version.

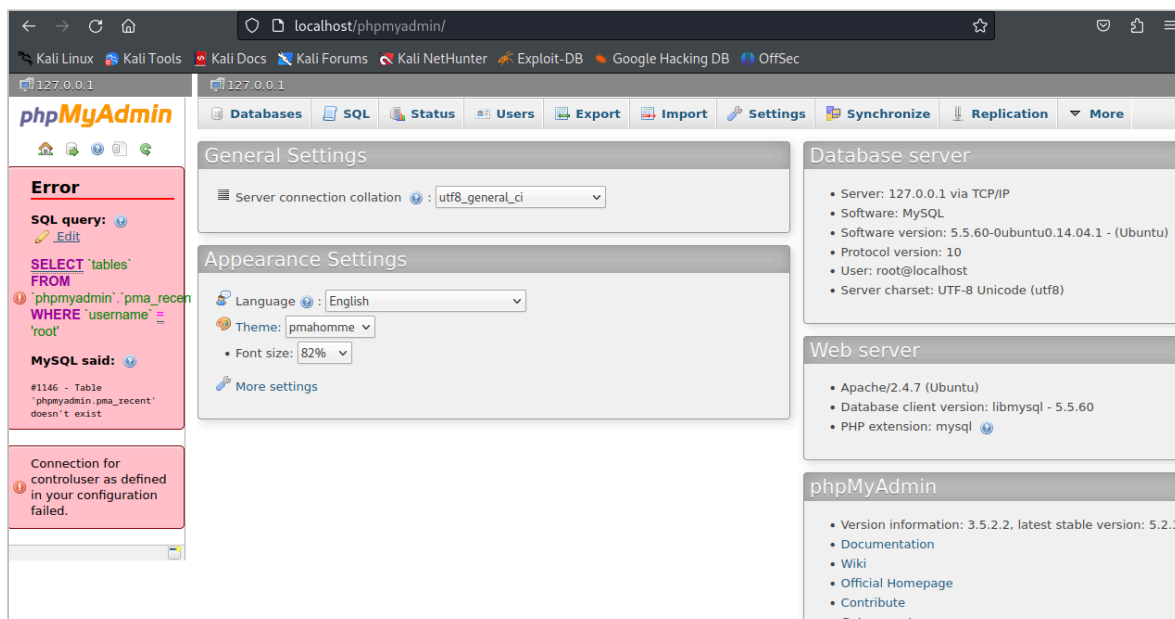


Figure 9 — phpMyAdmin accessible at /phpmyadmin/ revealing MySQL and version details

Finding 3: Verbose error messages

- **Issue:** The database error page (Figure 5) reveals internal file paths, the database name, and SQL hints.
- **How an attacker could use it:** Map the server's file structure and configuration, and use the details to plan SQL injection or file-inclusion attacks.
- **Possible impact:** Greatly accelerates further exploitation of the server.
- **Mitigation:** Set `display_errors = Off` in `php.ini` and configure generic custom error pages; log detailed errors server-side only.

Finding 4: Missing security headers and version disclosure

- **Issue:** The HTTP response lacks security headers (X-Frame-Options, Content-Security-Policy, X-Content-Type-Options) while advertising exact software versions (Apache 2.4.7, PHP 5.5.9).
- **How an attacker could use it:** Missing headers enable client-side attacks such as clickjacking; version banners allow instant lookup of known exploits.
- **Possible impact:** Attacks against site visitors, and faster targeted exploitation of the server.
- **Mitigation:** Add the missing security headers in the Apache configuration; hide versions with `ServerTokens Prod`, `ServerSignature Off`, and `expose_php = Off` in `php.ini`.

Part 5 — Attack Research and Proof of Concept: Directory Traversal

How it works

Web applications sometimes take a filename supplied by the user (for example `?page=home.php`) and load that file into the page. If the application does not validate this input, an attacker can use the sequence `../` ("go up one directory") to escape the web root and read arbitrary files anywhere on the server, such as `/etc/passwd` or application configuration files.

Typical attack process

- Identify a parameter that loads files (e.g., `page=`).
- Test it with `../` sequences, increasing the number until the target file is reached.
- Read sensitive files: password files, configuration files, source code.
- Use the information obtained for deeper attacks (credential theft, further exploitation).

Proof of Concept

The following URL was requested against the Mutillidae server:

```
http://localhost/index.php?page=../../../../etc/passwd
```

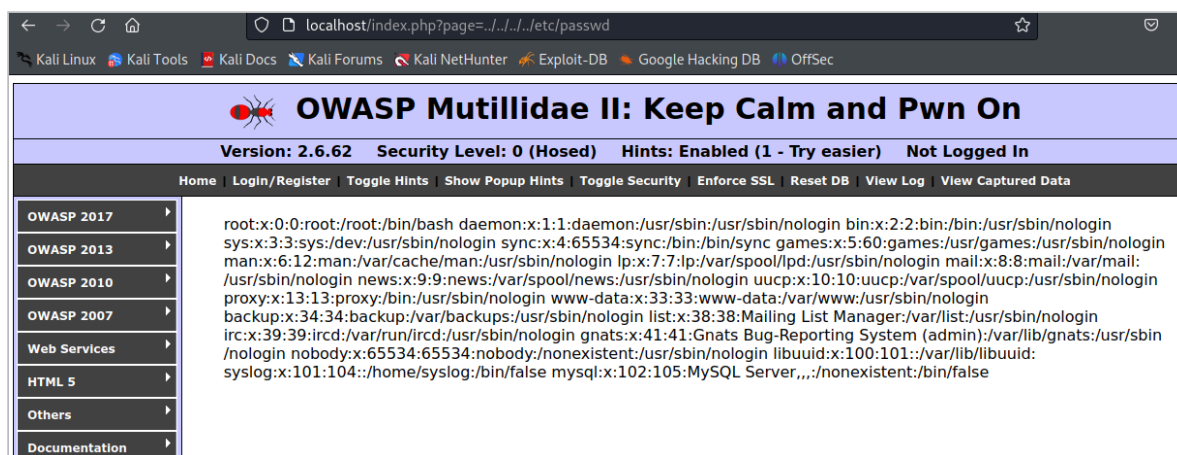


Figure 10 — Proof of concept — the contents of the server's `/etc/passwd` file displayed inside the web page

The server returned the full contents of `/etc/passwd` — the operating system's user account list — rendered inside the web page. This proves an attacker can read arbitrary server files with no authentication whatsoever, using only a browser.

Real-world incident

In 2021, a path traversal vulnerability in the Apache HTTP Server (CVE-2021-41773) was actively exploited in the wild. Attackers used `../` sequences to read files outside the web root, and on servers with certain

configurations the flaw even allowed remote code execution. Apache released an emergency patch, but the first fix was incomplete and a second vulnerability (CVE-2021-42013) had to be patched days later — a strong example of why input validation and patch management matter.

Business impact

Leaked configuration files and credentials can lead to a full server takeover and data breach. Consequences include regulatory fines under privacy laws, incident response and downtime costs, and serious reputational damage.

Detection opportunities

Web server access logs containing `../` or its encoded variants (e.g., `%2e%2e%2f`); Web Application Firewall (WAF) alerts on path traversal signatures; unusual file-access patterns by the web server process.

Prevention methods

Never pass user input directly to file operations; use a strict allow-list of permitted pages/files; run the web server in a container or chroot with minimal privileges so that even a successful traversal cannot reach sensitive files; keep all software patched.

Part 6 — Patch Management Assessment

Scenario: the web server software is outdated, security updates have not been applied for over a year, and no formal patch management process exists. The following plan uses the six standard stages.

Stage	Purpose	Why it is important	Example activity
1. Detect	Know what software and versions are running, and what vulnerabilities affect them	You cannot patch what you do not know about	Inventory Apache/PHP/MySQL versions; subscribe to vendor security mailing lists; run regular vulnerability scans (e.g., OpenVAS)
2. Assess	Prioritize which patches matter most	Not every patch is urgent; limited time must go to the highest risk first	Score each vulnerability with CVSS; verify whether the flaw actually affects the current configuration
3. Acquire	Obtain patches from trusted sources	Fake or tampered patches can install malware instead of fixing it	Download updates only from official vendor sites; verify checksums and digital signatures

4. Test	Make sure the patch does not break anything	A bad patch can take the website down or corrupt data	Apply the patch on a staging copy of the server and run regression tests before touching production
5. Deploy	Apply the patch to production safely	Poorly planned deployment causes outages and data loss	Patch during a scheduled maintenance window with a full backup and a documented rollback plan
6. Maintain	Keep the process running long-term	New vulnerabilities appear constantly; one-time patching is useless	Schedule monthly patch reviews, re-scan after each cycle, and document everything in a change log

Part 7 — Web Server Hardening Plan

Critical (address immediately)

#	Action	Expected security benefit	Risk reduction achieved
1	Remove /passwords/accounts.txt from the web root and reset all exposed passwords	Stolen credentials are invalidated; no passwords remain publicly accessible	Eliminates a confirmed, active account-compromise risk
2	Upgrade Apache 2.4.7, PHP 5.5.9, phpMyAdmin 3.5.2.2, and MySQL 5.5.60 to supported versions	Closes years of publicly known vulnerabilities (CVEs)	Removes trivially exploitable entry points
3	Restrict or remove /phpmyadmin/ (IP allow-listing + authentication, or uninstall)	The database admin interface is no longer exposed to the public	Prevents direct database takeover

High Priority (next maintenance window)

#	Action	Expected security benefit	Risk reduction achieved
4	Disable directory listing (<code>Options - Indexes</code>)	Attackers can no longer browse folder contents	Stops discovery of sensitive files
5	Hide version information (<code>ServerTokens Prod, ServerSignature Off, expose_php = Off</code>)	Attackers can no longer fingerprint exact software versions	Slows targeted exploitation and forces attackers to guess

6	Add security headers (X-Frame-Options, Content-Security-Policy, X-Content-Type-Options)	Browser-level protection for users against clickjacking and content sniffing	Reduces impact of client-side attacks such as XSS and clickjacking
7	Disable verbose errors (<code>display_errors = Off</code>) and use custom error pages	No more file path, database, or SQL information leaked to visitors	Removes a major reconnaissance shortcut
8	Fix robots.txt: remove sensitive entries; protect those directories with authentication instead	No public map of sensitive areas	Reduces targeted probing of admin/config paths

Long-Term Improvements

#	Action	Expected security benefit	Risk reduction achieved
9	Enforce HTTPS with TLS 1.2+ only	All traffic between users and the server is encrypted	Prevents credential and session theft via network sniffing
10	Deploy a Web Application Firewall and centralize log monitoring and alerting	Attacks such as directory traversal are blocked or detected in real time	Faster incident response and fewer successful attacks
11	Establish the formal patch management process from Part 6, with regular vulnerability scans and penetration tests	Security becomes continuous instead of a one-time effort	Keeps the server's overall risk low over time

Conclusion

This assessment showed how much information a misconfigured web server exposes without any active exploitation: exact software versions, operating system details, internal file paths, database names, a database administration panel, and even a plain-text password file. Simple reconnaissance — reading headers, checking robots.txt, browsing directories, and viewing page source — was enough to map the server and identify critical weaknesses. The proof of concept demonstrated that directory traversal can expose any file on the system through a web browser alone. Applying the patch management plan and the hardening recommendations in this report would significantly reduce the server's attack surface and protect it against the most common classes of web attacks.